Graduation project in Information Technology, 2025/2026

CANvisionNative

A diagnostic and validation platform for the CAN traffic of Battery Management Systems

Presented by

Mohamed Mootaz Ben Kraiem

Academic supervisor
Mrs Nadia Boulifa
ESPRIT

Corporate supervisor
Mr Hamda Guizani
Capgemini Engineering



The dashboard of the application

Summary

1 **Context**
The host company and how a CAN bus works

2 **Problem**
What is missing today and the existing tools

3 **Objectives and requirements**
The objectives, the method and the requirements

4 **Proposed solution**
Architecture, detection, models and explanation

5 **Demonstration**
The real application on real recordings

6 **Validation**
Two iterations on real recordings

7 **Limits**
What is not proven yet

8 **Conclusion**
Results and next steps

Part 1 of 8: Context

Who is the host company?

Capgemini Engineering, my host company

Capgemini engineering

The engineering and research and development branch of the Capgemini group, which offers consulting and engineering services to many industries, the automotive sector among them.

Automotive division, Capgemini Tunisia

Works on vehicle software systems and on embedded engineering.

Battery Management Systems team

Develops and validates the embedded software of battery management functions for electric and hybrid vehicles.

My job: BMS QA tester for BMW Group

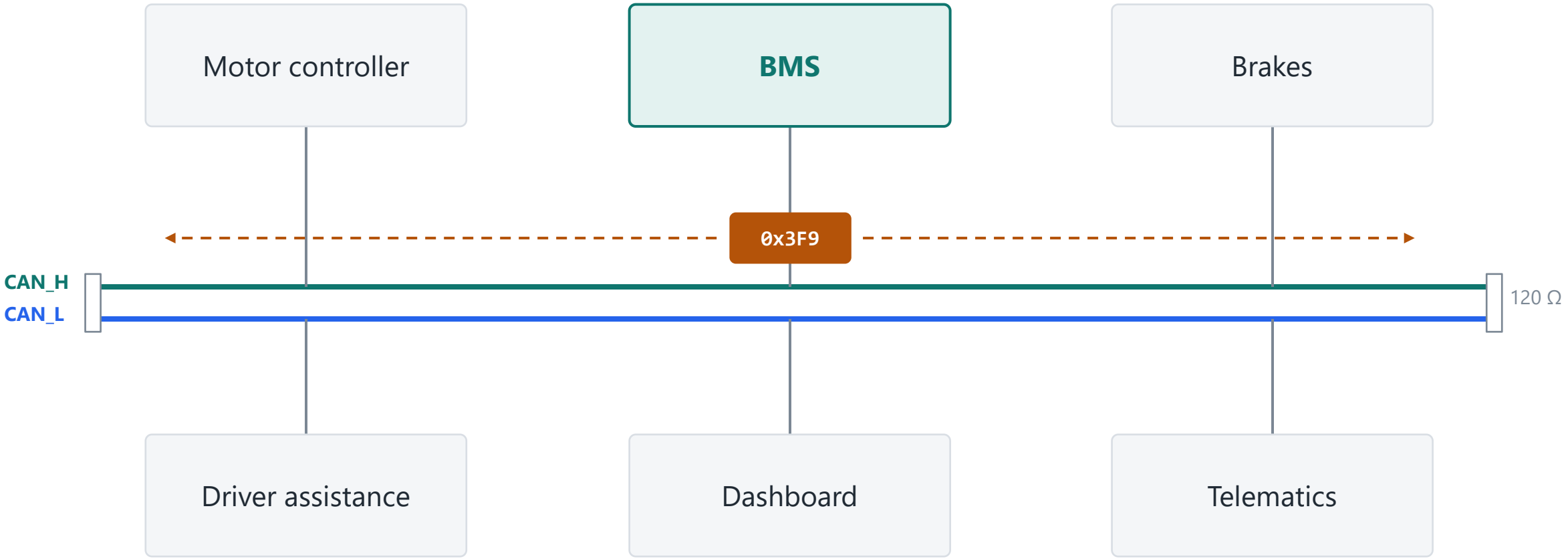
Work and study program since September 2024: full time, five days a week, with classes at ESPRIT on Saturdays.

CANvisionNative is not a team deliverable. I started it in February 2026, alongside my job, from the difficulties of my daily validation work.

Part 1 of 8: Context

How do the computers of a car talk to each other?

One bus shared by every computer of the car



Each ECU broadcasts short frames on one shared two-wire bus, and every node receives every frame.

A CAN frame does not say who sent it

Identifier
11 bits: what the frame is, and its priority

Length
number of data bytes

Data
up to 8 bytes; their meaning depends on the vehicle (DBC file)

CRC
error check



Same identifier, same bytes.
Nothing in the frame tells the receivers who sent it.

No sender identity
A frame does not say which ECU sent it.

No authentication
Any node can send any identifier.

No encryption
Every node reads every frame.

The bus cannot tell a genuine frame from a forged one, or from a faulty one.

CAN protocol: ISO 11898. Shown on real cars by Koscher et al. (2010) and Miller and Valasek (2015).

Part 2 of 8: Problem

What are the problems?

The open issues of daily BMS validation work

1	**No live view of the traffic**	Raw frames say what happened, not whether it should have happened.
2	**Weak replay**	Tools replay frames without judging them, or accept only a few log formats.
3	**Scattered tools**	An analyser, a diagnostic scanner and a spreadsheet, for one investigation.
4	**Alerts without explanation**	A bare number, with no cause and no next step.

Underneath all four: CAN has no security. A toolchain that does not watch the bus cannot notice a malicious frame any more than a faulty one. A real but secondary motivation.

Part 2 of 8: Problem

What already exists?

Four families of existing tools

CAN analysers

Good at
Capture, DBC decoding and replay. The professional standard (CANoe, CANalyzer).

Missing
No judgement on whether a frame is normal for this vehicle.

Intrusion detection research

Good at
Learns normal traffic and flags what deviates from it.

Missing
One signal, aims at attacks, rarely transfers to another vehicle, no explanation.

Open source viewers

Good at
Lightweight and open source: SavvyCAN, candump.

Missing
Little control over replay, no notion of abnormal.

Diagnostic scanners

Good at
Standard requests (OBD, UDS) to known ECUs.

Missing
Only see what they ask for, not the whole bus.

Each tool covers one part. None of them combines decoding, replay, live monitoring, detection and explanation in one workflow.

Problem statement

How can we give a BMS engineer one view of the CAN traffic, adapted to each vehicle, that explains its own alerts?

… and prove that it works on real recordings.

Problem statement

How can we give a BMS engineer one view of the CAN traffic, adapted to each vehicle, that explains its own alerts?

… and prove that it works on real recordings.

One view of the traffic
Answers
No live view, weak replay, scattered tools

Adapted to each vehicle
Answers
Models rarely transfer from one vehicle to another

Explains its own alerts
Answers
Alerts without explanation: a bare number, with no cause

The idea

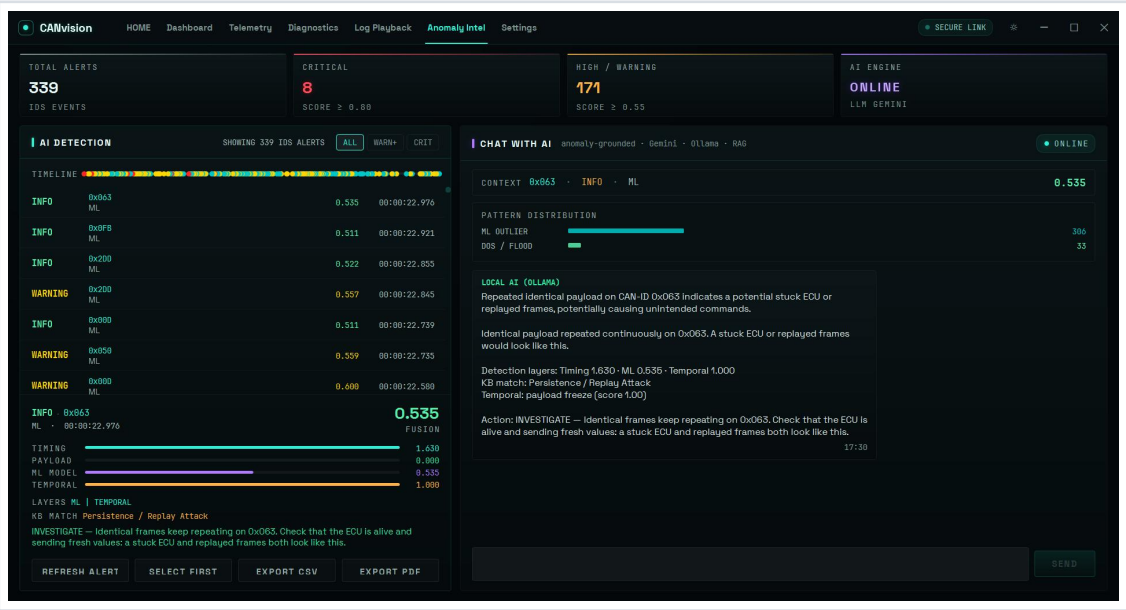
CANvisionNative

One platform where a BMS engineer replays, monitors and understands the CAN traffic of a vehicle.

One view
replay, live monitoring and detection together

Adapted to each vehicle
its own models and its own DBC file

Explains its own alerts
in plain language, next to the score



In the application: the alerts, their layer scores and the explanation

Part 3 of 8: Objectives and requirements

What are the objectives?

What are the objectives?

General objective

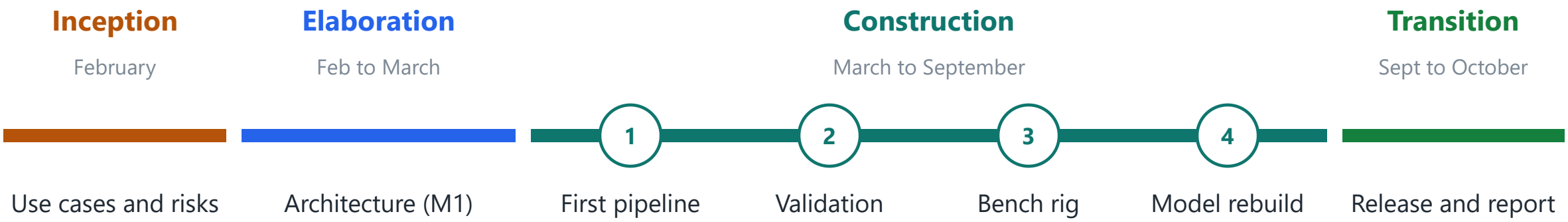
Design, build and validate a diagnostic platform for Battery Management Systems that gives an engineer one view of the CAN traffic, adapts to the vehicle being analysed, and explains its own alerts.

Specific objectives

1. A detection pipeline that combines a machine learning model with timing, payload and persistence checks
2. The common log formats (CSV, MF4, candump, Vector, PCAN), a live simulator and live capture
3. One model per vehicle, and in production only the models evaluated on a recording kept out of training
4. A desktop client to import a log, replay it, review the anomalies and export a report
5. An explanation layer, separate from detection, with rules and a language model
6. A validation on real recordings with reproducible evidence, and every defect found corrected

How did I organize the work?

The Unified Process



Why: iterative, driven by risks, architecture first.

Two unknowns at the start: how well a model generalises, and how many defects validation finds.

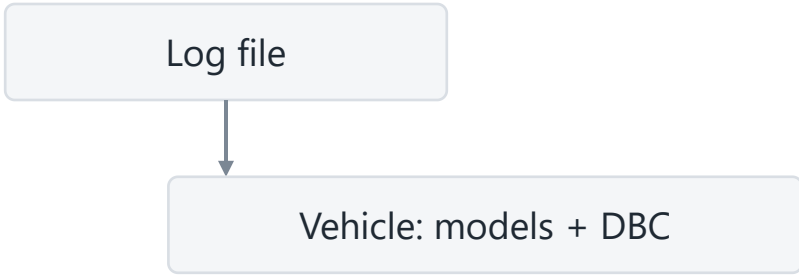
Part 3 of 8: Objectives and requirements

What must the platform do?

What must the platform do?

1 **Import a recorded log**
CSV, MF4, Vector .asc, candump .log, PCAN .txt into one frame stream

2 **Choose the vehicle**
Its own models score the frames; its DBC file decodes them



What must the platform do?

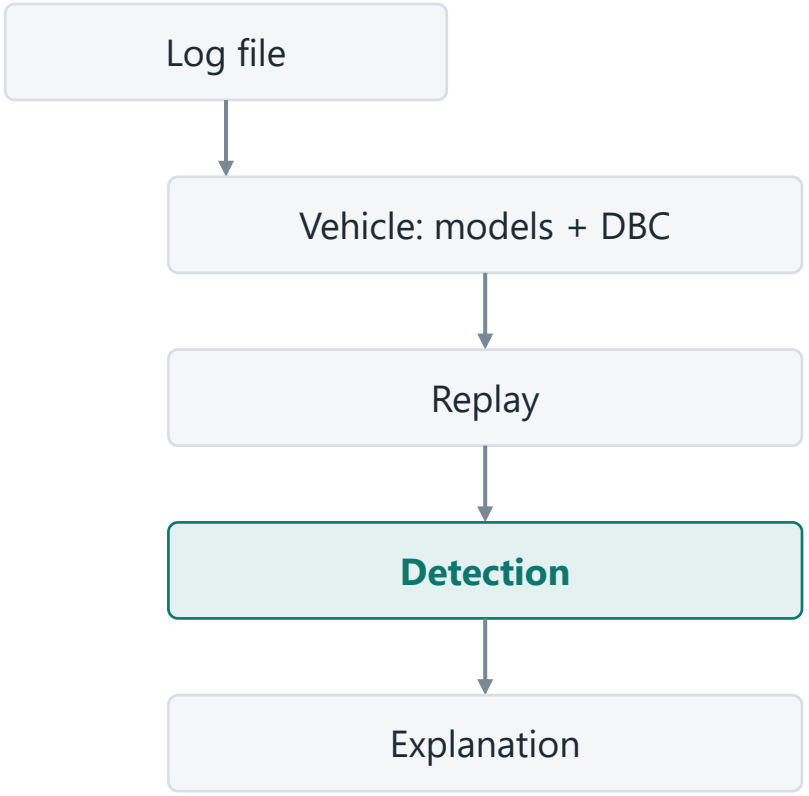
1 **Import a recorded log**
CSV, MF4, Vector .asc, candump .log, PCAN .txt into one frame stream

2 **Choose the vehicle**
Its own models score the frames; its DBC file decodes them

3 **Replay the log**
From 0.5× to 32× real time, with progress

4 **Detect anomalies**
Score each frame; report what cannot be scored, with the reason

5 **Explain an alert**
Dominant signal, likely cause, what to check



What must the platform do?

1 **Import a recorded log**
CSV, MF4, Vector .asc, candump .log, PCAN .txt into one frame stream

2 **Choose the vehicle**
Its own models score the frames; its DBC file decodes them

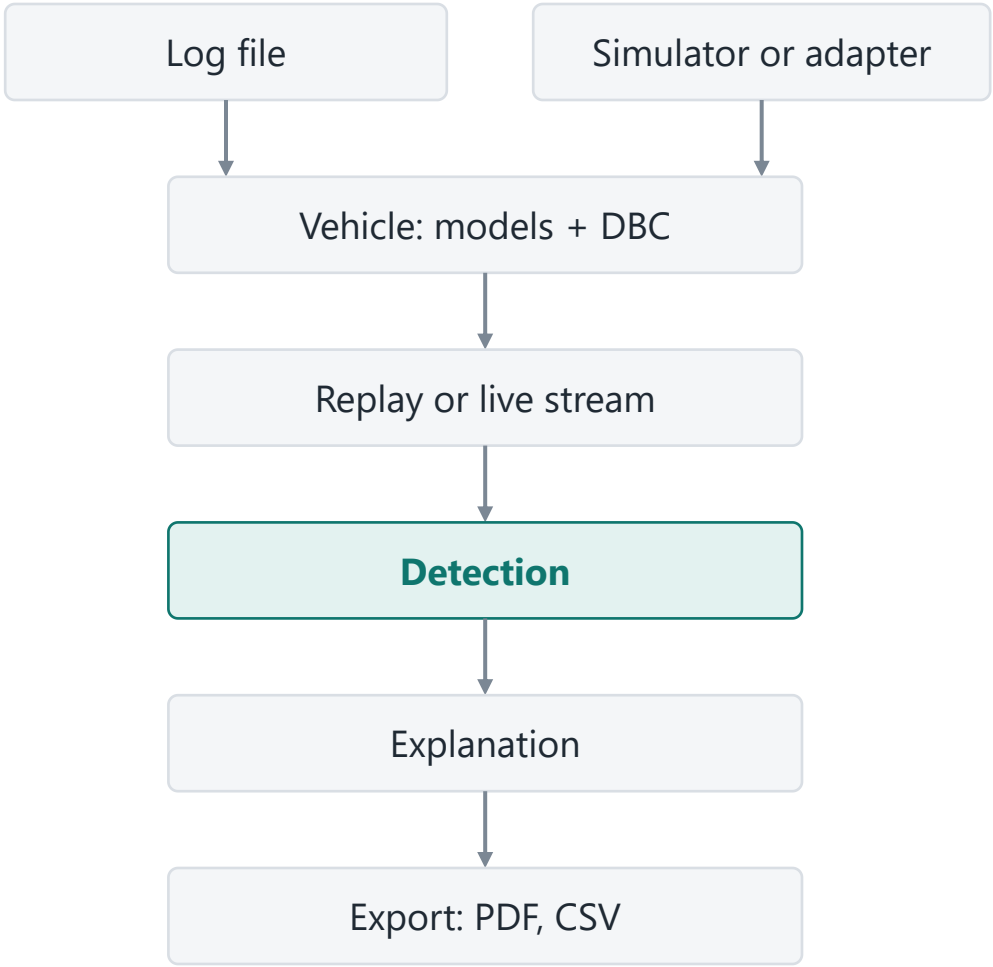
3 **Replay the log**
From 0.5× to 32× real time, with progress

4 **Detect anomalies**
Score each frame; report what cannot be scored, with the reason

5 **Explain an alert**
Dominant signal, likely cause, what to check

6 **Go live**
Simulator, or a CAN adapter through python-can; record a session

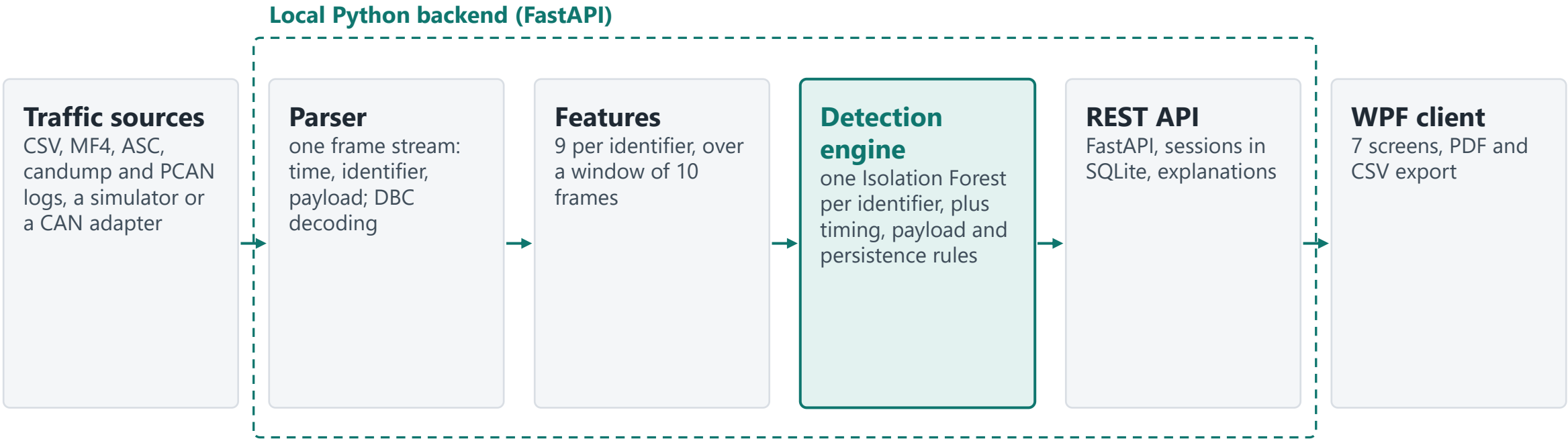
7 **Export the results**
CSV file and PDF report



Part 4 of 8: Proposed solution

What is the proposed solution?

Architecture: the way the data flows



Two programs on one machine: a local Python backend and a Windows client, talking over HTTP.

How one frame is judged

CAN frame
from the bus or the log

→

Features
9 per identifier, window of 10

→

Isolation Forest
the model of this identifier

→

Four layers
timing 0.35, payload 0.35, model 0.20, persistence 0.10

→

Fused score
weighted sum of the four layers

→

Decision
alert if the score reaches 0.40

Example: a normal frame	**fused score 0.17**	**Normal: nothing happens**
Example: an unusual frame	**fused score 0.82**	**Alert, severity HIGH**

Unscored frames are reported, never hidden. A frame with no model (warmup, sparse, unmodelled or unseen identifier) is counted as unresolved, with its reason.

Offline analysis of a whole recording: same features and models; each frame is flagged when the score of its model passes 0.5.

One model per CAN identifier

Frames of one identifier

here 0x3F9, in time order

```
35.000  4E 00 00 00
35.200  4E 00 00 00
35.400  4E 00 00 00
35.600  4E 00 00 00
35.800  4E 00 00 00
```

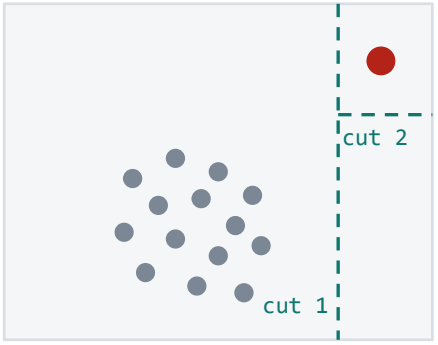
9 features

Timing ×3
time between frames, its rolling mean and std

Payload ×6
byte change, its rolling mean, std and max; changed bytes; entropy

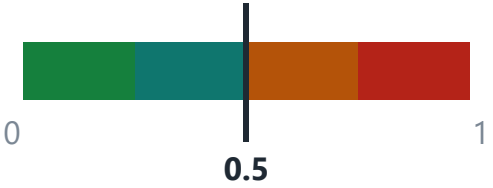
over a window of 10 frames

Isolation Forest



200 trees. An unusual frame is isolated in very few cuts.

A score from 0 to 1



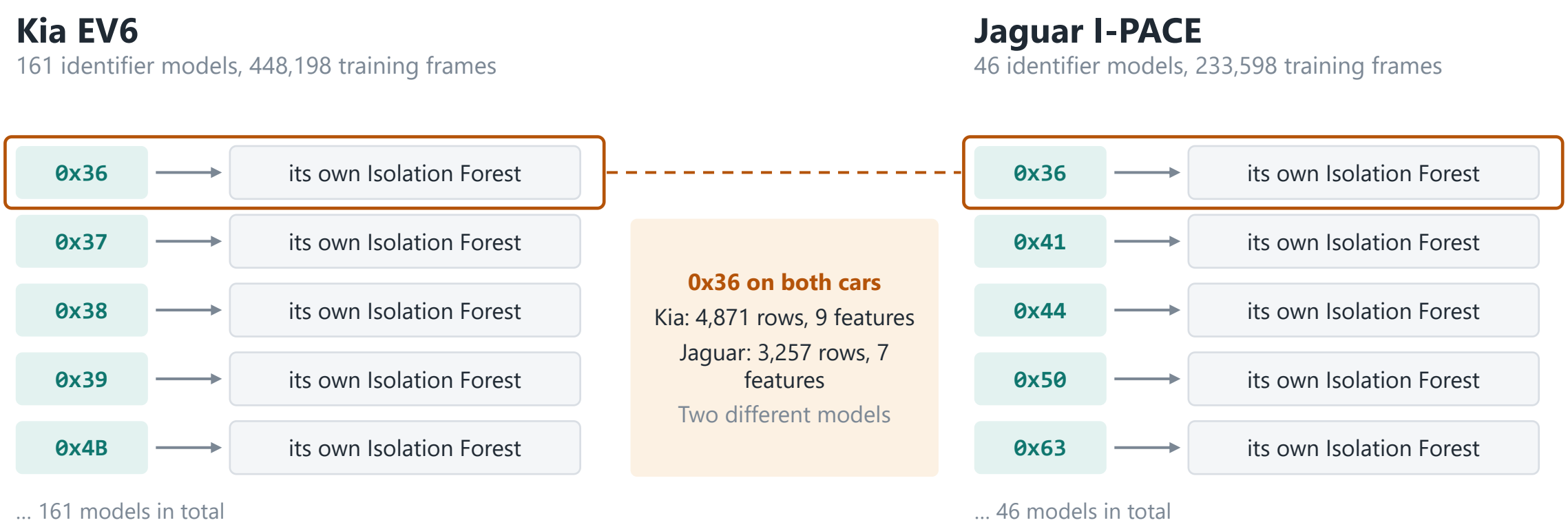
The threshold, 0.5, is set so that 2 % of the training rows pass it. Above it, the frame is flagged.

At least 1,000 training rows per identifier, or no model

Not scored, and said so: warmup, sparse, unmodelled or unseen identifier

The timing features carry 64 % of the splits

Why one model per vehicle?



CAN traffic depends on the vehicle, so the models are vehicle specific.

Explaining an alert

Alert
`0x063` `score 0.535`
from the detection engine

→

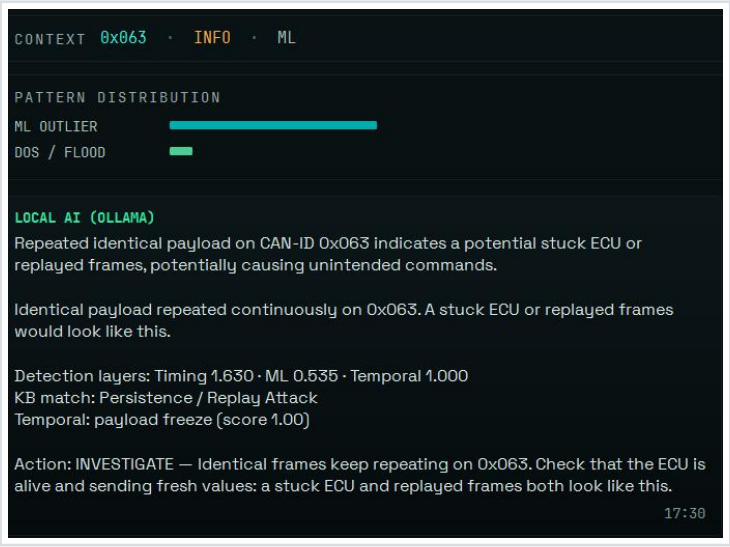
Context
decoded signals, layer scores, temporal pattern, knowledge base

→

Explainer
Gemini; if it cannot be reached, a local Ollama model; if neither answers, rules

→

Diagnosis
dominant signal, likely cause, what to check



In the application: the explanation of the alert on 0x063

AI explains.

The detection engine decides.

The detection engine sets the scores, the flags and the severities.

The explanation is words only: it never changes a score, a flag or a severity, and its numbers are checked against the alert.

Part 5 of 8: Demonstration

Can we see it working?



The real application



Part 6 of 8: Validation

How do we know it works?

How do we know it works?

Iteration 2: the first pipeline on a one hour Kia EV6 recording (97,741 frames)

15,945

alerts, 16.3 % of the frames

Instead of raising the threshold, we attributed every alert to the layer that dominated it:



- **Machine learning 61.42 %** wrong model, wrong driving state
- **Timing 31.51 %** irregular UDS polling
- **Payload 7.07 %** real sensor variation

Six silent defects

1. The selected vehicle model was never used for scoring
2. Driving state: 99.93 % "parking" for an hour of driving
3. Diagnostic fields dominated the cause shown to the analyst
4. Replay formats other than CSV bypassed the common parser
5. A polling timer stayed stopped after an import
6. The explanation cache returned another session's text

None of them crashed. Each appeared only when the output was compared with a fact known independently, such as the car being driven.

How do we know it works?

Iteration 2: six corrections, the same recording, the same threshold

11,971

alerts after the corrections, against 15,945 before



Saturated model score (≥ 0.999)	77.6 to 78.6 % → **0.0 %**
Replay throughput	≈ 143 frames/s → **≈ 194.6 frames/s**
Driving state	99.93 % parking → **99.92 % driving**

Six defects, six corrections

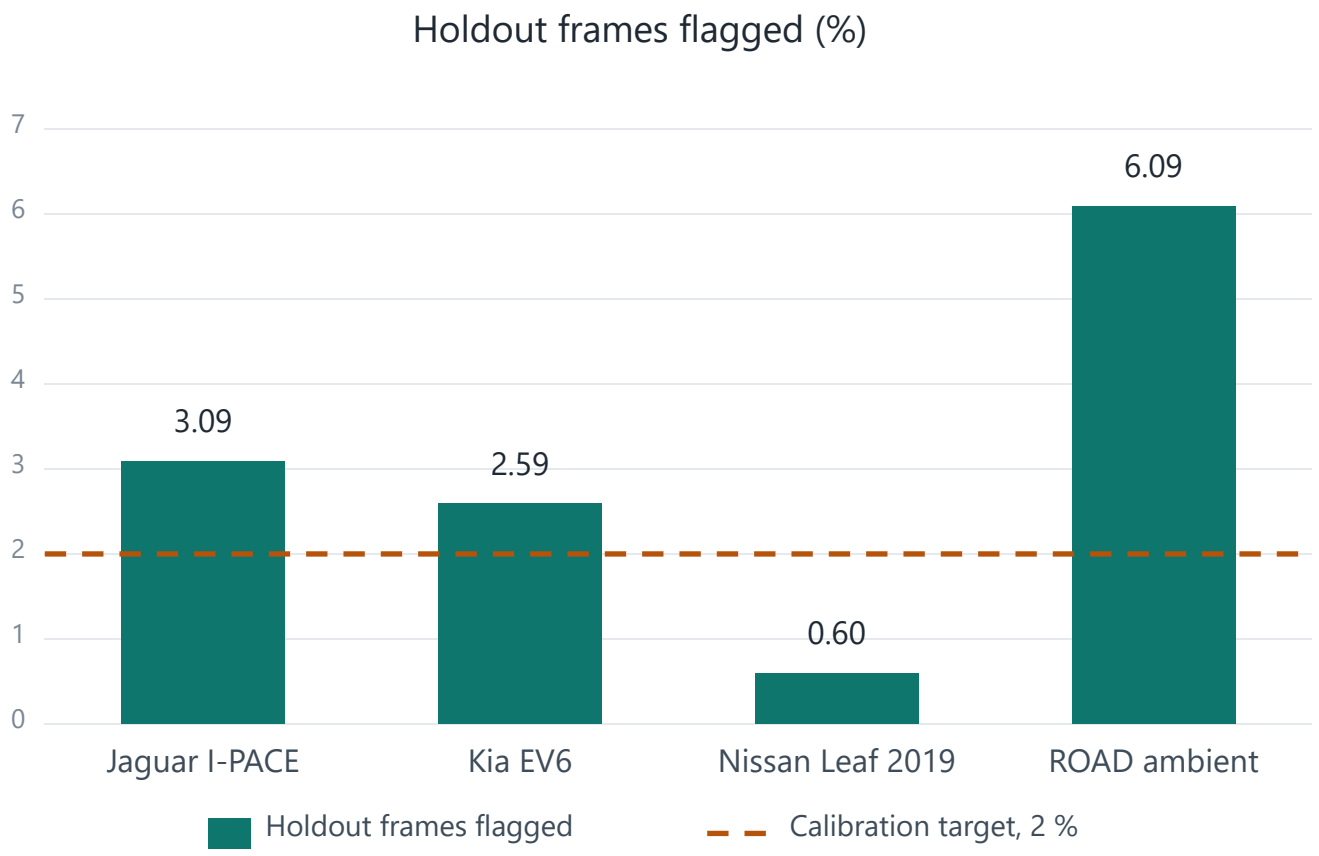
- ✓ The selected vehicle model was never used for scoring
- ✓ Driving state: 99.93 % "parking" for an hour of driving
- ✓ Diagnostic fields dominated the cause shown to the analyst
- ✓ Replay formats other than CSV bypassed the common parser
- ✓ A polling timer stayed stopped after an import
- ✓ The explanation cache returned another session's text

The threshold was never touched. The design stayed fragile, so I rebuilt every model with a stricter pipeline in iteration 4.

Counts measured after the four data-driven fixes (1 to 4); fixes 5 and 6 were in the client and the explanation cache.

How do we know it works?

Iteration 4: the final models on recordings they never saw



296 identifier models
for 4 vehicles

3,436,061 training frames
1,130,500 holdout frames, no overlap

ROAD at 6.09 %: ID 0x193 never changes in training but changes on the highway. A coverage gap, not a threshold problem.

Held back: MG ZS EV, 43.6 % flagged, with 78 % of its holdout on identifiers it never saw.

No attack labels: a flagged rate on unlabelled recordings is not a false positive rate, a precision or a recall.

Part 7 of 8: Limits

What are its limits?

Four limits, stated plainly

Live capture	**Tested on a bench rig only, not on a vehicle.** The transport of live frames is validated; detection on a live vehicle bus is not measured.
Live performance	**About 200 to 260 frames per second.** Enough for a slow bus such as the Nissan Leaf (about 100 frames/s), not for buses that carry thousands.
Holdout data	**No attack labels.** We report flagged rates, not a detection rate, a precision or a recall.
Vehicle coverage	**Rare identifiers get no model.** Jaguar I-PACE: 82.6 % of the holdout frames scored. One model held back (MG ZS EV).

Each limit is measured and stated in the report.

Part 8 of 8: Conclusion

What can we conclude?

Delivered, and what I learned

Delivered

- ✓ A Windows client and a Python backend: recorded logs, a live simulator and live capture
- ✓ 4 vehicle models, 296 identifier models, evaluated on 1,130,500 frames they never saw
- ✓ A training pipeline that refuses a model failing its evaluation
- ✓ Explanations in plain language, kept apart from detection
- ✓ A bench rig with its firmware, and 318 + 86 automated tests

What I learned

The most costly defects were the silent ones: none of them crashed.

A holdout is only meaningful when the split is made by recording, not by frame.

A flagged rate on unlabelled data is not a detection rate.

"A detection system is only as trustworthy as the evidence behind each of its numbers."

What comes next?

1 **Live capture on a real vehicle**
The first step: it closes milestone M2.

2 **Labelled attacks**
Evaluate the final models on recordings with labelled attacks, such as the ROAD dataset, to measure what they detect as well as what they flag.

3 **A faster live path**
Score several frames at once or use fewer trees, with each change measured on the holdout recordings.

4 **More recordings per vehicle**
A holdout for every vehicle, and a model for more of the rare identifiers.

CANvisionNative

From CAN frames to explainable diagnostics.

Diagnose, monitor, detect, explain and validate

Thank you for your attention. **Questions?**

esprit Se former autrement | Capgemini engineering

Backup

How should it behave?

Performance
Offline: hundreds of thousands of frames in a few minutes. Live: keep up with the bus.

Reliability
A defect is fixed with evidence, never by tuning a threshold until the numbers look better.

Usability
No machine learning background needed to read an alert and its explanation.

Format independence
Every log format enters the pipeline as the same frame stream.

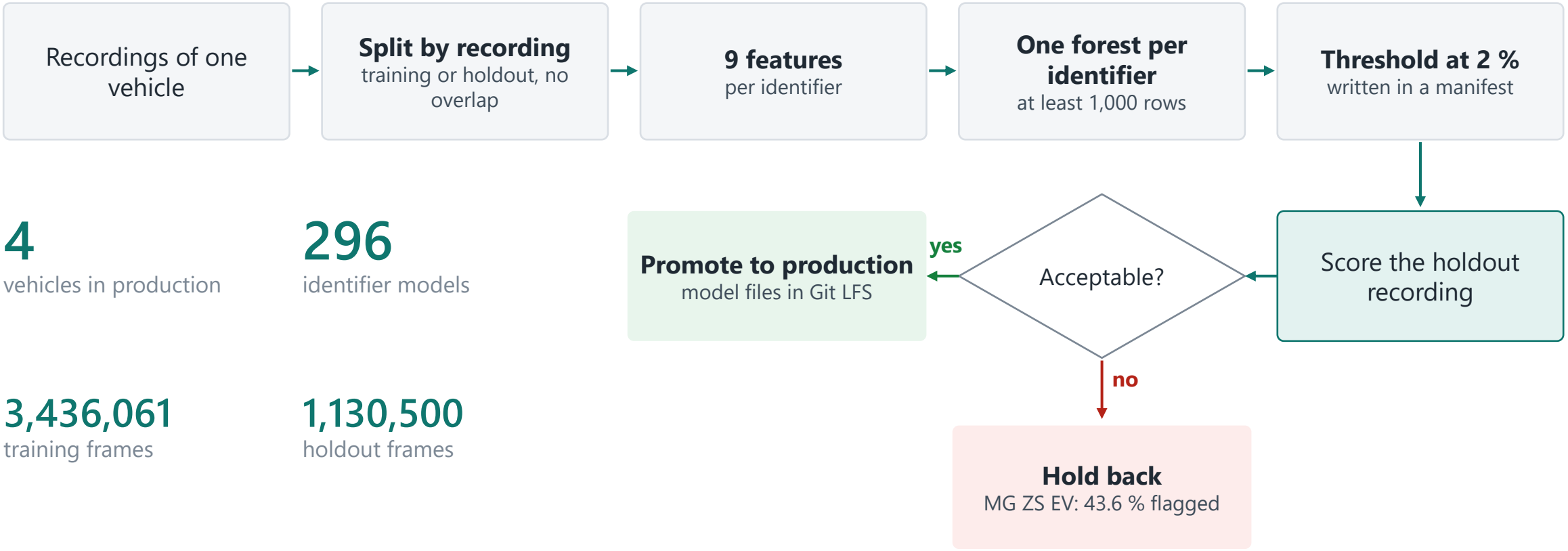
Extensibility
A new vehicle needs only a DBC file and a model folder that passes the promotion checks.

Graceful degradation
No language model reachable: a rule based explanation instead of a failure.

Platform: Windows 10 or 11 with .NET Framework 4.8, a local Python 3.11 backend, and a python-can adapter only for live capture

Backup

From recordings to a model in production



4
vehicles in production

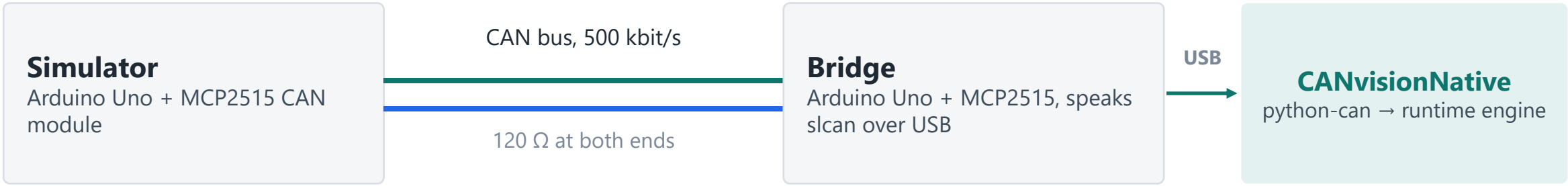
296
identifier models

3,436,061
training frames

1,130,500
holdout frames

Backup

Live capture without a vehicle: the bench rig



What the simulator sends

`0x316`	speed, 60 km/h	every 20 ms
`0x3F9`	state of charge, 78 %	every 200 ms
`0x666`	should not exist	every 20 s, from 35 s

About 55 frames per second

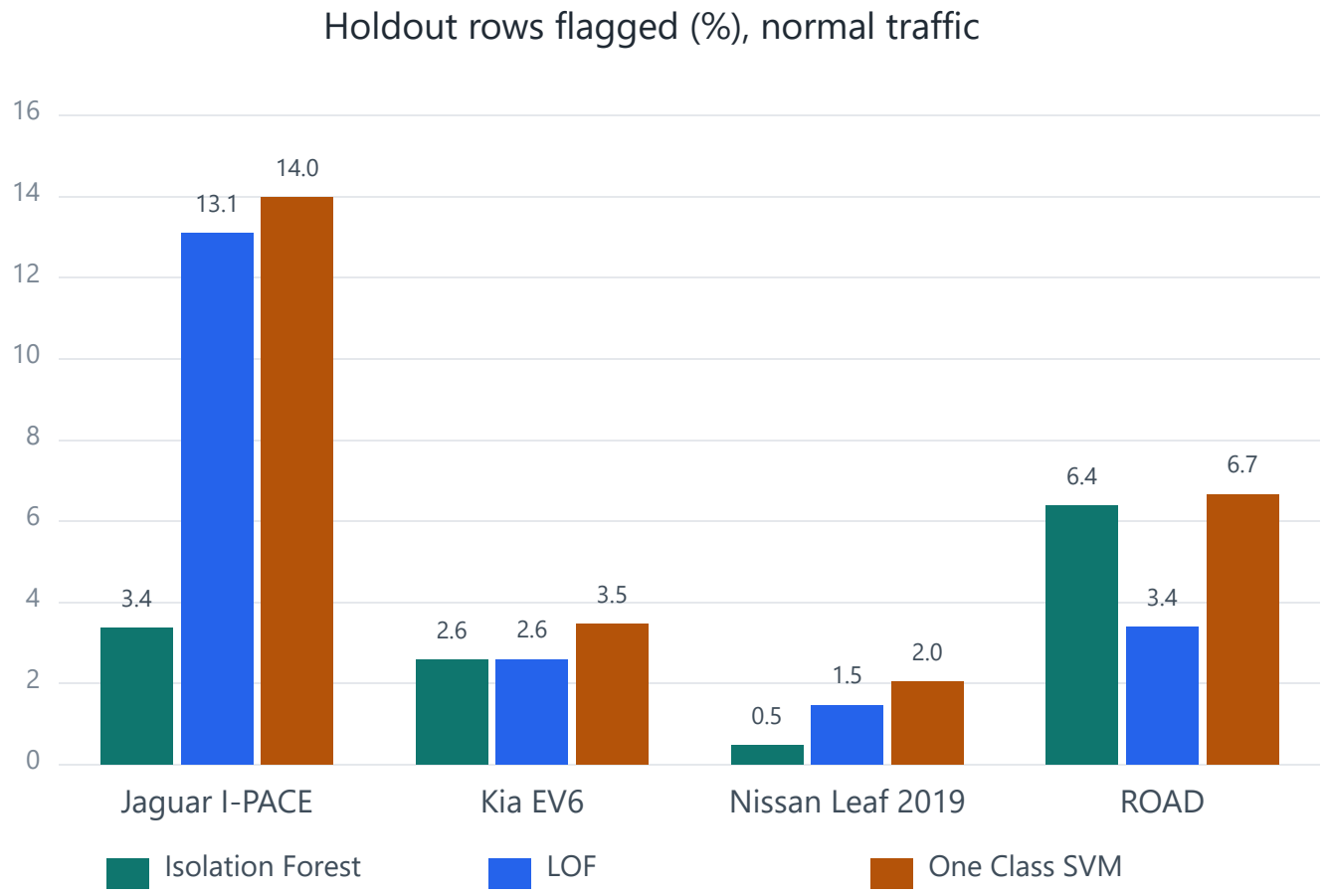
What it shows, and what it does not

It tests the transport of live capture, from the bus to the runtime engine.

It does not measure detection: the traffic is synthetic and much simpler than a car's. No vehicle was available.

Backup

Why Isolation Forest?



Mean share flagged

Isolation Forest	LOF	One Class SVM
3.2 %	5.1 %	6.5 %

Lowest worst case: 6.4 % for Isolation Forest, against 13.1 % and 14.0 %.

The price: it is the slowest. 23.4 ms to score a frame against 0.97 and 0.42 ms; 138 s of training against 41 and 17 s.

We kept it: the offline analysis, the main use of the platform, needs few and even false alarms more than speed.

Same split, same 9 features, same training rows (at most 5,000 per identifier), same threshold rule. No labels.

Backup

Speed, live capture and automated tests

Speed

≈ 24 ms

per model call

Live path: 200 to 260 frames/s

Keeps up with the Nissan Leaf (about 100 frames/s), not with the Jaguar I-PACE (about 3,000)

Offline: 491,969 frames in 187 s

Live capture on the rig

16,304

frames, then one malformed serial line closed the session

Fixed: the bad line is skipped and logged; the link then held at about 55 frames/s

Fixed: an unplugged rig kept showing old data; the link is now lost after 5 s without frames

Automated tests

318 + 86

backend tests (pytest) and desktop tests (xUnit)

Run after every change: parsers, manifests, model status, scoring, unresolved frames, calibration, PDF report